

RAG-Powered Chatbot for Japan Travel Assistance: A Hybrid Q/A Retrieval Approach with Vector Search

José Julián Gutiérrez Badilla

Software Engineer, CodeCademy

joseguba1604@gmail.com

May 2026

Abstract. This paper presents the design, implementation, and evaluation of a Retrieval-Augmented Generation (RAG) chatbot for Japan travel assistance, developed as the capstone project for the AGAI-03 Agentic AI Development Bootcamp. The system uses the official Japan National Tourism Organization (JNTO) website as its knowledge base, scraping 55 curated pages via a Playwright-based web scraper with site-specific CSS selectors. A synthetic Q/A dataset of 338 question-answer pairs was generated using Claude Sonnet 4.6 with adaptive prompting calibrated to page content length. The pairs and 836 document chunks are indexed in ChromaDB using the `all-MiniLM-L6-v2` sentence-transformer model. A two-stage hybrid retrieval strategy applies direct Q/A matching when cosine similarity exceeds 0.75, falling back to semantic document search otherwise. The chatbot interface, implemented in Streamlit, provides source citations, retrieval mode indicators, and confidence scores. Evaluation on test queries yields similarity scores of up to 0.946 for high-confidence Q/A matches.

Keywords: Retrieval-Augmented Generation; Large Language Models; Vector Database; ChromaDB; Web Scraping; Playwright; Streamlit; Japan Tourism.

Contents

1	Introduction	4
1.1	Objectives	4
1.2	Document Structure	4
2	Chosen Website and Justification	4
2.1	Selected Data Source	4
2.2	Justification	5
2.2.1	Content Richness and Diversity	5
2.2.2	Ethical Scraping Compliance	5
2.2.3	Real-World Relevance	5
3	Scraping Methodology and Challenges	5
3.1	Technology Selection	5
3.2	Scraping Pipeline	6
3.3	Challenges Encountered	6
3.3.1	JavaScript-Rendered Content	6
3.3.2	Cookie Consent Popup	6
3.3.3	URL Structure Inconsistency	6
3.3.4	Content Selector Specificity	6
3.4	Dataset Statistics	7
4	Q/A Generation Strategy	7
4.1	Motivation	7
4.2	Adaptive Pair Generation	7
4.3	Prompt Engineering	7
4.4	Dataset Statistics	8
5	System Architecture	8
5.1	Overview	8
5.2	Component Inventory	9
5.3	Knowledge Base Structure	9
6	Hybrid Retrieval Architecture	10
6.1	Motivation	10
6.2	Two-Stage Retrieval	10
6.2.1	Stage 1 — Q/A Collection Search	10
6.2.2	Stage 2 — Document Chunk Fallback	10
6.3	Embedding Strategy	10
6.4	Conversation Memory	10
7	Chatbot Interface	11
7.1	Implementation	11

8	Limitations and Future Improvements	12
8.1	Current Limitations	12
8.1.1	Static Knowledge Base	12
8.1.2	Coverage Scope	12
8.1.3	Threshold Sensitivity	12
8.1.4	Session-Only Memory	12
8.2	Future Improvements	12
9	Learnings	13
9.1	Technical Learnings	13
9.1.1	JavaScript Rendering in Modern Web Scraping	13
9.1.2	CSS Selector Precision	13
9.1.3	Embedding Strategy for Q/A Retrieval	13
9.2	AI/RAG Conceptual Learnings	13
9.2.1	RAG vs. Pure Generation	13
9.2.2	Synthetic Q/A as Semantic Compression	13
9.2.3	Stateless APIs and Application-Layer Memory	13

1. Introduction

The emergence of large language models (LLMs) has opened new possibilities for conversational information retrieval. However, general-purpose LLMs are prone to *hallucination* — generating plausible but factually incorrect responses — particularly for domain-specific or time-sensitive queries. Retrieval-Augmented Generation (RAG) [1] addresses this limitation by grounding model responses in retrieved, authoritative source documents.

This project implements a RAG-based travel assistant for Japan, using the Japan National Tourism Organization (JNTO) official website as the authoritative knowledge source. The choice of domain is motivated by several factors: (1) travel information is highly specific and frequently updated, making hallucination particularly harmful; (2) user queries are naturally suited to Q/A retrieval; and (3) the JNTO website provides rich, well-structured content covering diverse aspects of travel to Japan.

1.1 Objectives

1. Design and implement a web scraper capable of extracting clean, structured content from a modern JavaScript-rendered website.
2. Generate a high-quality synthetic Q/A dataset using prompt-engineered LLM calls.
3. Build a hybrid retrieval pipeline combining Q/A exact matching with semantic document search.
4. Develop a production-quality chatbot interface with source citation and retrieval transparency.

1.2 Document Structure

Section 2 describes the chosen data source and its justification. Section 3 details the scraping methodology and technical challenges. Section 4 presents the Q/A generation strategy. Section 5 provides the system architecture. Section 6 explains the hybrid retrieval mechanism. Section 7 showcases the chatbot interface. Section 8 discusses limitations and future work. Section 9 reflects on technical and professional learnings.

2. Chosen Website and Justification

2.1 Selected Data Source

The knowledge base is sourced from the **Japan National Tourism Organization (JNTO)** official English-language travel guide at <https://www.japan.travel/en/us/>. JNTO is the government-designated body responsible for international tourism promotion, and its website constitutes the most authoritative publicly available English resource for travelers to Japan.

Table 1: Data source metadata

Attribute	Value
Website	<code>https://www.japan.travel/en/us/</code>
Publisher	Japan National Tourism Organization (JNTO)
Language	English (en/us locale)
Content types	Destinations, experiences, planning guides, articles
<code>robots.txt</code>	Allow: / — full crawling permitted

2.2 Justification

2.2.1 Content Richness and Diversity

The JNTO website covers all facets of travel to Japan: 47 regional destinations, food and beverage culture, traditional arts, outdoor activities, practical planning (visas, transportation, budgeting), and seasonal guides. This breadth enables the chatbot to respond to a wide variety of user queries without requiring supplementary sources.

2.2.2 Ethical Scraping Compliance

The `robots.txt` file specifies `Allow: /`, permitting automated crawling of all English-language pages. Only three paths are disallowed: `/admin/`, `/travel-directory/`, and `/jp/`. The scraper applies a 1.5-second delay between requests and uses a descriptive `User-Agent` header.

2.2.3 Real-World Relevance

A Japan travel assistant addresses a genuine user need. Travelers frequently seek authoritative, specific answers about destinations, transportation, customs, and entry requirements — a use case that demonstrates RAG’s value for domain-specific question answering.

3. Scraping Methodology and Challenges

3.1 Technology Selection

The scraper was implemented using **Playwright** (Python API) combined with **BeautifulSoup4** for HTML parsing. Playwright was selected over the simpler `requests+BeautifulSoup` approach after discovering that `japan.travel` renders its primary content via client-side JavaScript.

Table 2: Scraping technology stack

Tool	Version	Purpose
Playwright	1.x	Headless Chromium browser for JS execution
BeautifulSoup4	4.12.x	HTML parsing and content extraction
python-dotenv	1.0.x	Configuration management
tqdm	4.66.x	Progress visualization

3.2 Scraping Pipeline

The final scraper implements the following pipeline:

1. **URL Management:** A curated list of 55 high-value URLs is maintained in `data/urls.txt`. Automated link crawling was abandoned after discovering the site's inconsistent locale URL structure.
2. **Cookie Dismissal:** The GDPR cookie consent popup is dismissed programmatically on the first page load. The Playwright browser context persists this state for all subsequent requests.
3. **Content Rendering:** Playwright waits for the `networkidle` event before HTML extraction, ensuring complete JavaScript execution.
4. **Targeted Extraction:** The CSS selector `div.content-main-wrapper` targets the article container. Only `h1-h4`, `p`, and `li` elements are extracted.
5. **Quality Filtering:** Pages are discarded if content is fewer than 200 characters, the title indicates a soft 404, or content contains unrendered JS template literals.

3.3 Challenges Encountered

3.3.1 JavaScript-Rendered Content

The initial `requests+BeautifulSoup` implementation returned fewer than 100 characters of usable content for most sub-pages. Migrating to Playwright and awaiting `networkidle` resolved this, yielding rich content across all pages.

3.3.2 Cookie Consent Popup

The GDPR dialog appeared on every page load and caused the extractor to capture identical popup content across all pages. Dismissing the popup via Playwright's `page.click()` API before extraction resolved the issue.

3.3.3 URL Structure Inconsistency

The site uses `/en/us/` for the homepage and `/en/` for all sub-pages. Automated crawling constructed incorrect URLs with the `/us/` prefix, producing 100% soft 404 responses. A curated URL list and validation function resolved this.

3.3.4 Content Selector Specificity

Generic tag-based extraction captured navigation labels, language selectors, and cookie policy text. The site-specific selector `div.content-main-wrapper`, identified through DOM inspection, consistently encloses only article content.

3.4 Dataset Statistics

Table 3: Final scraped dataset statistics

Metric	Value
Total pages scraped	55
Total word count	$\approx 35,000$ words
Word count range	142 – 1,521 words per page
Average words per page	≈ 636
Output format	JSON (<code>url, title, content, word_count</code>)

4. Q/A Generation Strategy

4.1 Motivation

A synthetic Q/A layer improves retrieval precision for common, well-defined queries. Pre-computing answers to likely user questions allows the system to return high-confidence responses without LLM synthesis from raw chunks, reducing both latency and hallucination risk for queries matching known patterns.

4.2 Adaptive Pair Generation

The number of pairs generated per page adapts to content length to maximize quality and avoid hallucination on information-sparse pages:

Table 4: Adaptive Q/A pair generation thresholds

Word Count	Pairs	Rationale
< 200	2	Insufficient content for diverse questions
200–400	3	Basic topic coverage
400–700	5	Moderate coverage
700–1200	7	Broad thematic coverage
> 1200	9	Maximum useful diversity

4.3 Prompt Engineering

The prompt was designed to produce structured JSON output for reliable parsing:

```

1 You are building a Q/A dataset for a Japan travel chatbot.
2 Given the following travel content, generate exactly {N} question-
  answer pairs.
3
4 Rules:
5 - Questions must be natural, specific, and directly answerable from the
  text
6 - Answers must be factual, concise (1-3 sentences), and based ONLY on
  the provided content

```

```

7 - Cover different aspects of the content (don't repeat similar
   questions)
8 - Questions should sound like real traveler queries
9 - Do NOT include questions about links, images, or website navigation
10
11 Content title: {title}
12 Content: {content[:3000]}
13
14 Respond ONLY with a valid JSON array, no explanation, no markdown:
15 [{"question": "...", "answer": "..."}, ...]
```

Listing 1: Q/A generation prompt (simplified)

4.4 Dataset Statistics

Table 5: Q/A dataset statistics

Metric	Value
Total Q/A pairs	338
Average pairs per page	6.1
Generation model	claude-sonnet-4-6 (Anthropic)
Generation time	≈90 seconds
Output file	data/qa_dataset.csv

5. System Architecture

5.1 Overview

The system operates in two phases:

Ingestion phase (offline): Data collection and knowledge base construction. Executed once before deployment. Comprises the scraper, Q/A generator, and vector store builder.

Query phase (real-time): User query processing and response generation. Executed on every user message. Comprises the hybrid retriever, Claude API call, and Streamlit rendering.

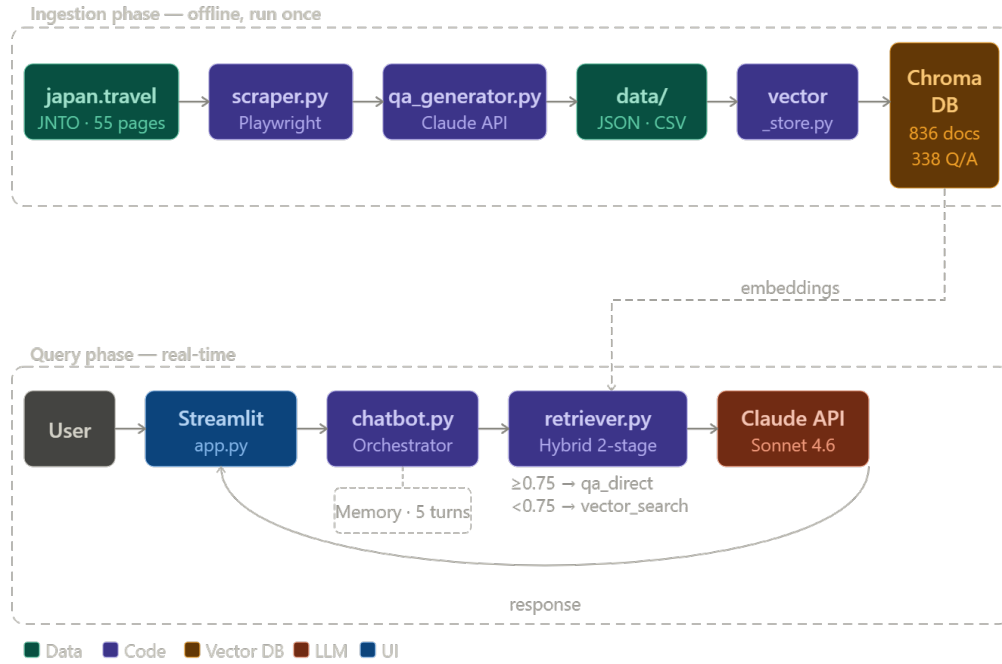


Figure 1: RAG pipeline architecture — ingestion and query phases.

5.2 Component Inventory

Table 6: System components and responsibilities

Component	File	Responsibility
Web Scraper	<code>src/scraper.py</code>	Playwright-based scraper with targeted CSS selectors.
Q/A Generator	<code>src/qa_generator.py</code>	Claude API calls with adaptive pair counts.
Vector Store	<code>src/vector_store.py</code>	Document chunking, embedding generation, ChromaDB indexing.
Hybrid Retriever	<code>src/retriever.py</code>	Two-stage retrieval with configurable similarity threshold.
Chatbot Core	<code>src/chatbot.py</code>	Retrieval orchestration, prompt construction, memory management.
UI	<code>app.py</code>	Streamlit interface with source citations and session management.

5.3 Knowledge Base Structure

ChromaDB maintains two logically separate collections:

documents: 836 chunks from 55 scraped pages. Chunk size: 512 characters, overlap: 80 characters. Metadata: `url`, `title`, `chunk index`.

qa_pairs: 338 question-answer pairs. Questions are embedded to match user query semantics; answers are stored as metadata.

6. Hybrid Retrieval Architecture

6.1 Motivation

Two retrieval paradigms exist for RAG systems: Q/A matching offers high precision for queries resembling pre-indexed questions but fails for open-ended queries; semantic document search generalizes to arbitrary queries but increases hallucination risk. The hybrid approach applies Q/A matching when confidence is sufficient and document search otherwise, optimizing both precision and coverage.

6.2 Two-Stage Retrieval

6.2.1 Stage 1 — Q/A Collection Search

The user query q is embedded using f_θ (all-MiniLM-L6-v2):

$$\mathbf{v}_q = f_\theta(q) \in R^{384}$$

Cosine similarity is computed against all 338 stored question vectors:

$$\text{sim}(q, q_i) = \frac{\mathbf{v}_q \cdot \mathbf{v}_{q_i}}{\|\mathbf{v}_q\| \cdot \|\mathbf{v}_{q_i}\|}$$

If $\max_i \text{sim}(q, q_i) \geq \tau = 0.75$, the stored answer is returned directly (qa_direct mode).

6.2.2 Stage 2 — Document Chunk Fallback

When no Q/A pair achieves similarity $\geq \tau$, the top $k = 3$ document chunks are retrieved and concatenated as context C for the LLM (vector_search mode).

6.3 Embedding Strategy

Questions (not answers) are indexed in qa_pairs because user queries are semantically analogous to questions. This design consistently yields high similarity scores (0.75–0.95) for queries matching stored question intent.

Table 7: Retrieval configuration and observed performance

Query	Mode	Score
“What airports serve Tokyo?”	qa_direct	0.94
“How to travel by Shinkansen?”	vector_search	0.59
“Atmosphere walking through Gion?”	vector_search	0.45
“Do I need visa for Japan?”	qa_direct	0.93

6.4 Conversation Memory

A sliding window of the last $n = 5$ conversation turns is prepended to each Claude API call, enabling anaphora resolution across turns without unbounded token growth.

7. Chatbot Interface

7.1 Implementation

The interface is implemented in Streamlit and provides: (1) a threaded chat window; (2) source citations with retrieval mode badge, confidence bar, matched Q/A pair, and hyperlinked source; (3) a sidebar with knowledge base statistics and sample questions; and (4) session export as plain text.

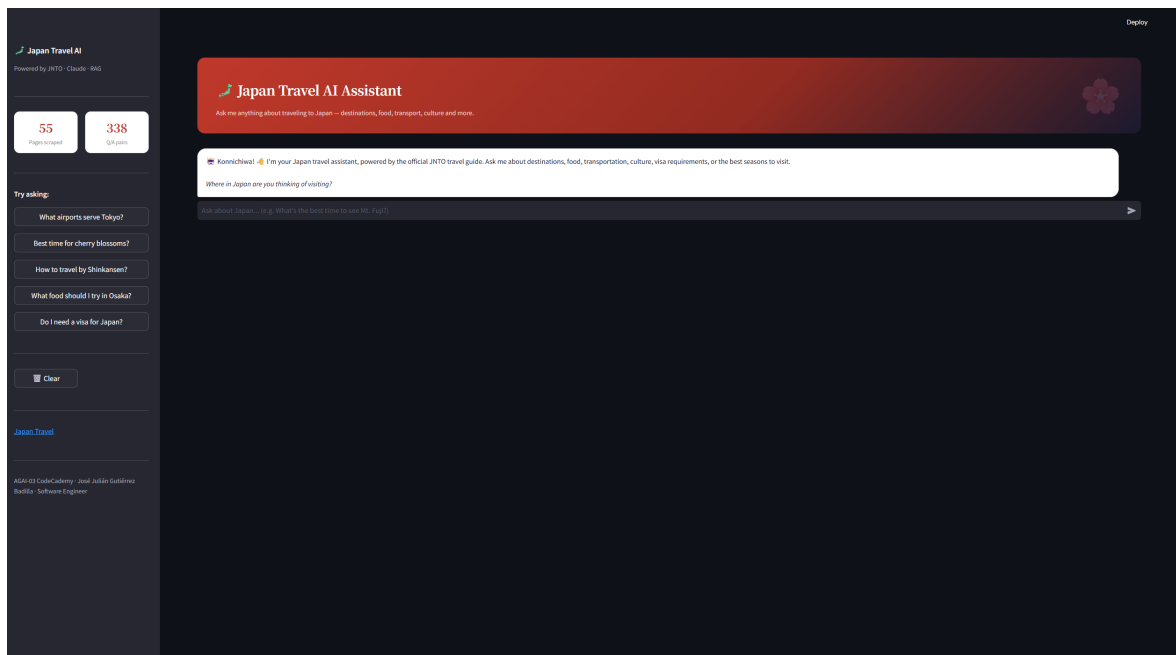


Figure 2: Main Streamlit interface showing the chat area and sidebar statistics.

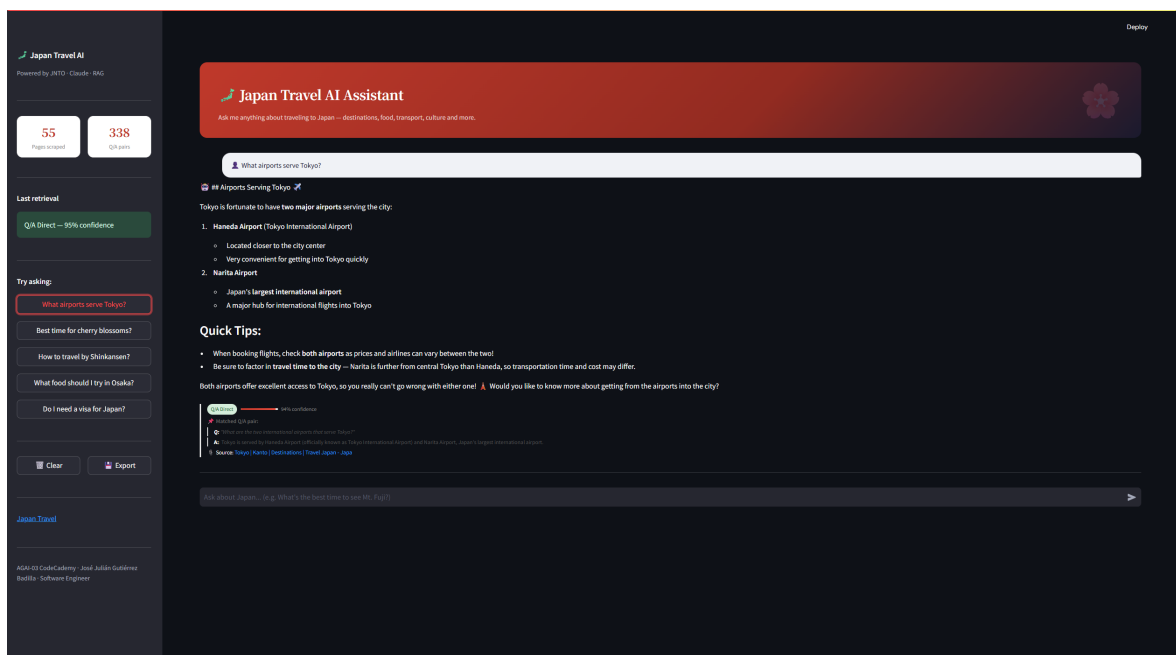


Figure 3: Q/A Direct response (0.946 confidence) with matched pair and source citation.

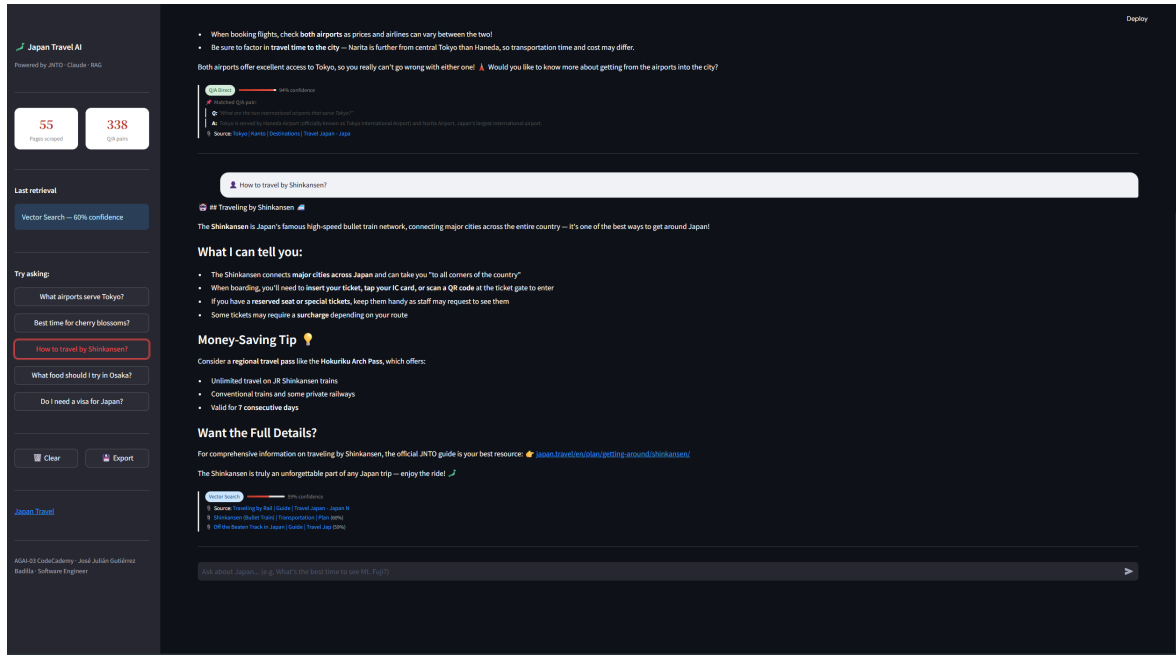


Figure 4: Vector Search mode with multiple source citations.

8. Limitations and Future Improvements

8.1 Current Limitations

8.1.1 Static Knowledge Base

The dataset reflects a single point in time. Travel information changes frequently — visa regulations, entry requirements, and seasonal events. The system has no mechanism for detecting or updating stale information.

8.1.2 Coverage Scope

The 55-page corpus covers major destinations and topics but omits many niche areas. Queries outside the scraped corpus yield lower-confidence `vector_search` responses that may be incomplete.

8.1.3 Threshold Sensitivity

The threshold $\tau = 0.75$ was set empirically. Optimal calibration requires a labeled evaluation set, which was outside the scope of this project.

8.1.4 Session-Only Memory

Conversation memory exists only within a single Streamlit session with no cross-session persistence.

8.2 Future Improvements

1. **Automated re-scraping:** Periodic scraping runs via GitHub Actions to maintain knowledge base currency.
2. **Re-ranking:** Cross-encoder re-ranking after initial retrieval to improve precision for ambiguous queries.

3. **Evaluation:** Labeled test set with RAGAS metrics for retrieval accuracy and answer faithfulness.
4. **Streaming responses:** Token-by-token output via `st.write_stream()` for improved perceived responsiveness.
5. **Deployment:** Streamlit Community Cloud for a publicly accessible portfolio URL.

9. Learnings

9.1 Technical Learnings

9.1.1 JavaScript Rendering in Modern Web Scraping

The migration from `requests+BeautifulSoup` to Playwright was the most consequential decision of this project. Modern websites universally rely on client-side rendering, making headless browser automation a fundamental skill for production-grade data collection. The `networkidle` strategy proved more reliable than fixed delays.

9.1.2 CSS Selector Precision

Site-specific selector targeting produces dramatically cleaner results than generic tag-based extraction. Identifying the correct selector required DOM inspection — a skill transferable to any scraping project.

9.1.3 Embedding Strategy for Q/A Retrieval

Embedding questions (not answers) aligns the retrieval space with user query intent. This design consistently produced high similarity scores for queries matching stored question patterns, validating the design decision empirically.

9.2 AI/RAG Conceptual Learnings

9.2.1 RAG vs. Pure Generation

The project demonstrated why domain-specific RAG outperforms zero-shot generation. Retrieval from an authoritative source eliminates hallucination for facts that LLMs would otherwise fabricate (specific visa fees, rail pass coverage, operating hours).

9.2.2 Synthetic Q/A as Semantic Compression

Generating Q/A pairs before indexing applies semantic compression: 35,000 words of variable-quality scraped text become 338 clean, directly answerable pairs. This pre-computation improves both response quality and retrieval latency.

9.2.3 Stateless APIs and Application-Layer Memory

LLM APIs are inherently stateless. Effective conversational AI requires the application layer to reconstruct context on every request. The sliding-window approach balances contextual coherence against token cost — a fundamental trade-off in production LLM applications.

References

- [1] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401.
- [2] Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019. arXiv:1908.11164.
- [3] Chroma. (2023). *ChromaDB: The AI-native open-source embedding database*. <https://www.trychroma.com/>
- [4] Anthropic. (2024). *Claude: AI Safety and Research*. <https://www.anthropic.com/>
- [5] Microsoft. (2023). *Playwright for Python*. <https://playwright.dev/python/>
- [6] Streamlit Inc. (2023). *Streamlit: The fastest way to build data apps*. <https://streamlit.io/>